

EECS 482

Lab 5: Semaphores & Deadlocks



Ashwani Gautam

Yesterday at 08:14

I: Explain us deadlock and we'll hire you

Me: Hire me and I'll explain it to you

Administrivia

Today: 02/14

Project 2

Due 06/20

~1 week

Midterm

06/24

~1.5 weeks

Agenda

1. Deadlock
2. Project 2 Debugging
3. Midterm Review

— — —

Deadlock

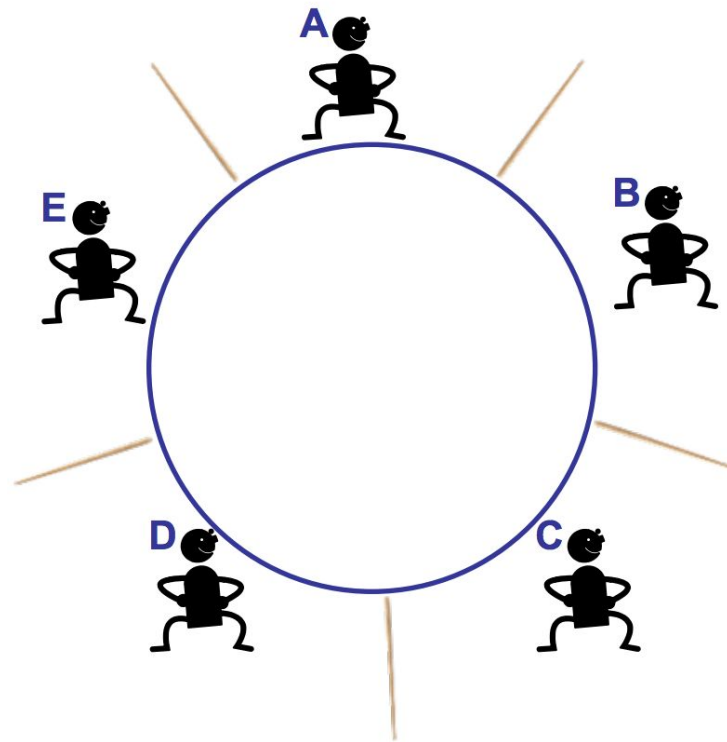
Conditions for deadlock

— — —

- **Limited resources**
 - Not enough to serve all threads at once
- **No preemption**
 - Cannot force threads to give up control
- **Hold-and-wait**
 - Holding resource while waiting for other
- **Cyclical chains of requests**
 - e.g. dining philosophers

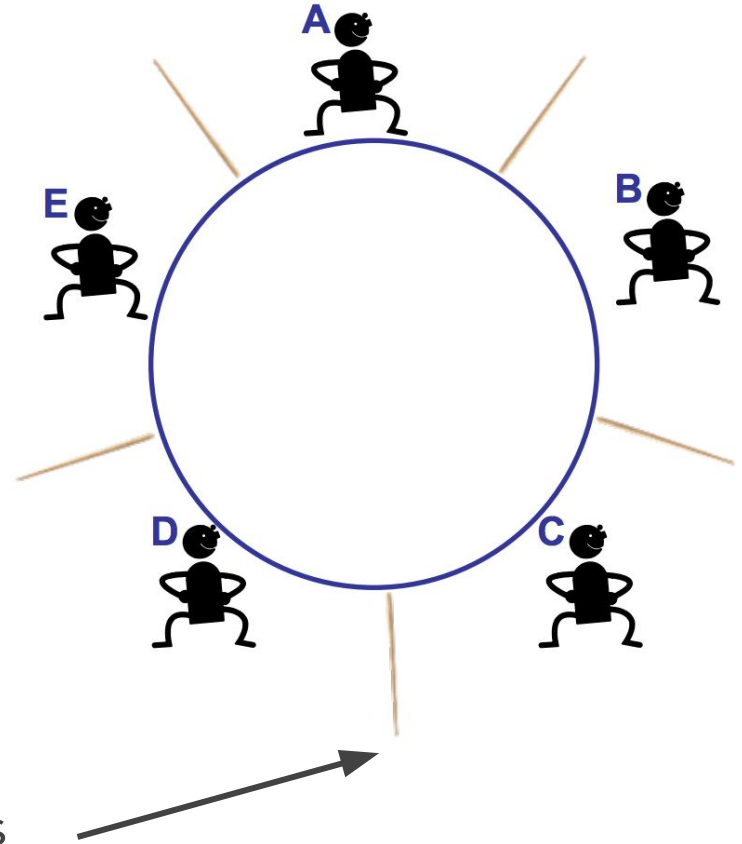
Dining philosophers

```
dine() {  
    waitFor(rightStick);  
    pickUp(rightStick);  
    waitFor(leftStick);  
    pickUp(leftStick);  
    monch();  
}
```



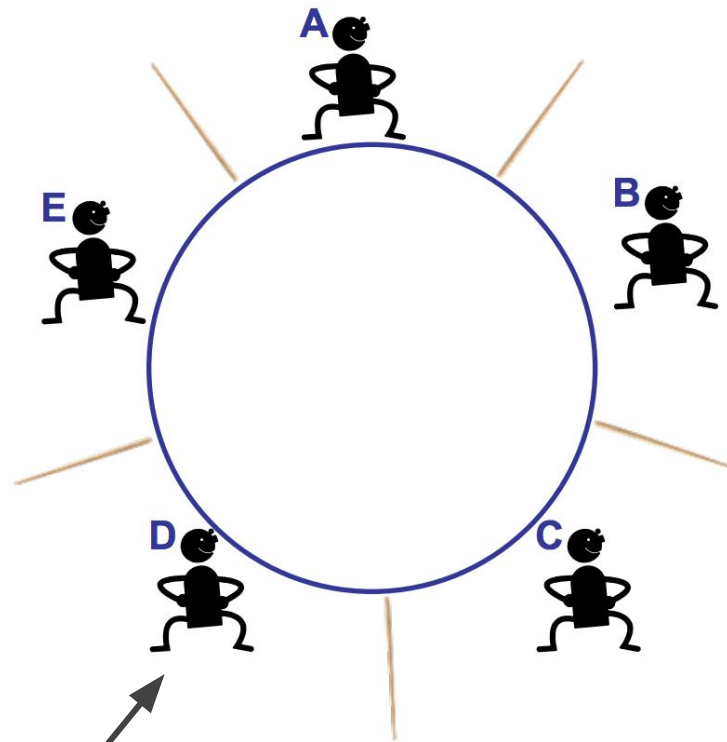
Dining philosophers

```
dine() {  
    waitFor(rightStick);  
    pickUp(rightStick);  
    waitFor(leftStick);  
    pickUp(leftStick);  
    monch();  
}
```



Dining philosophers

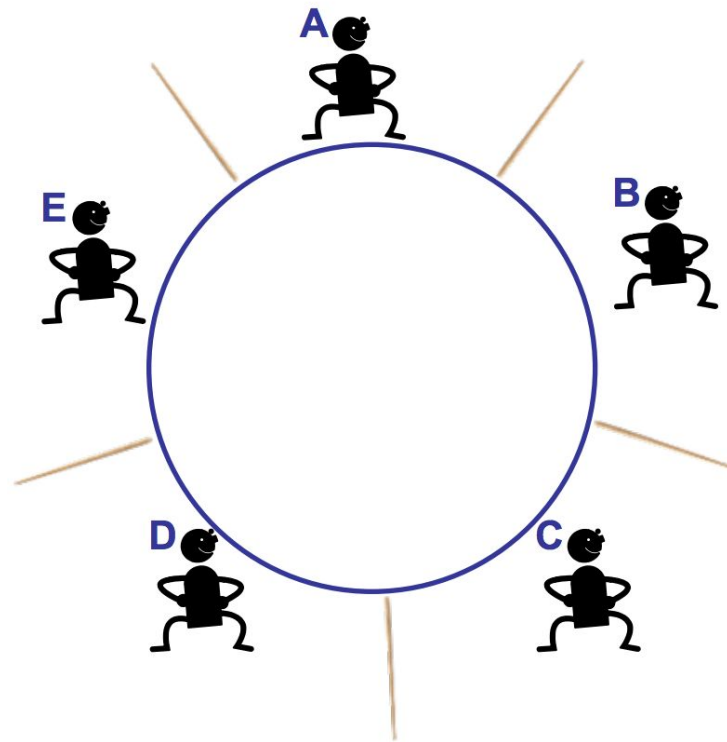
```
dine() {  
    waitFor(rightStick);  
    pickUp(rightStick);  
    waitFor(leftStick);  
    pickUp(leftStick);  
    monch();  
}
```



no preemption
while eating

Dining philosophers

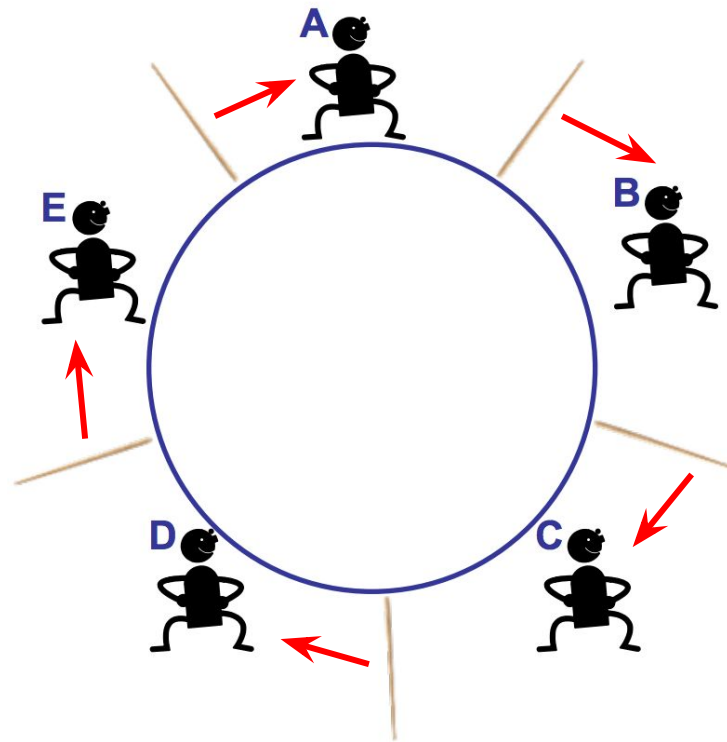
```
dine() {  
    waitFor(rightStick);  
    pickUp(rightStick);  
    waitFor(leftStick);  
    pickUp(leftStick);  
    monch();  
}
```



hold-and-wait

Dining philosophers

```
dine() {  
    waitFor(rightStick);  
    pickUp(rightStick);  
    waitFor(leftStick);  
    pickUp(leftStick);  
    monch();  
}
```



Cyclical acquisitions

Avoiding deadlock

— — —

We could **detect** deadlock:

- e.g., detect cycles in resource graph
- *Preempt* naughty threads

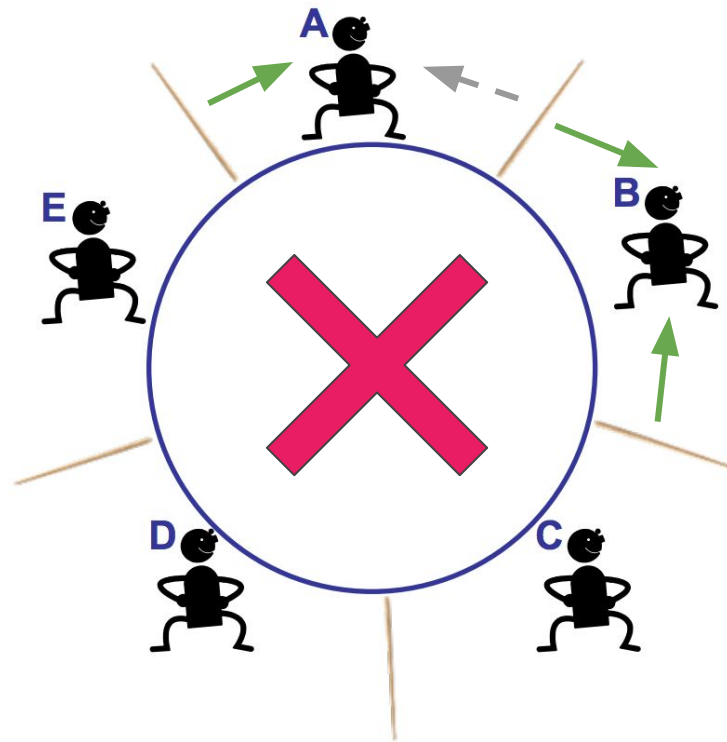
We could **prevent** deadlock:

- Prevent *hold-and-wait*
- Prevent *cyclical acquisitions*

Preventing hold-and-wait

Two approaches:

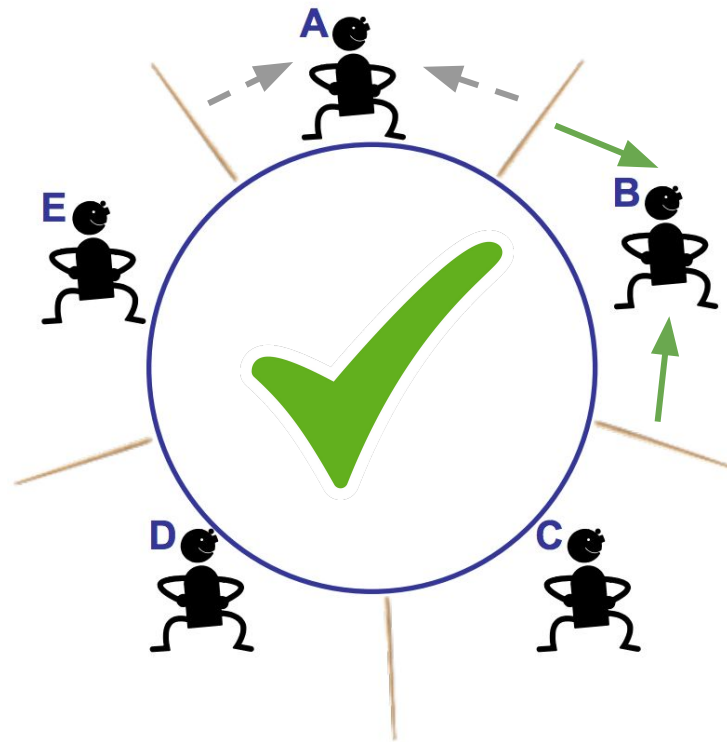
1. Threads grab all resources atomically, or none at all.



Preventing hold-and-wait

Two approaches:

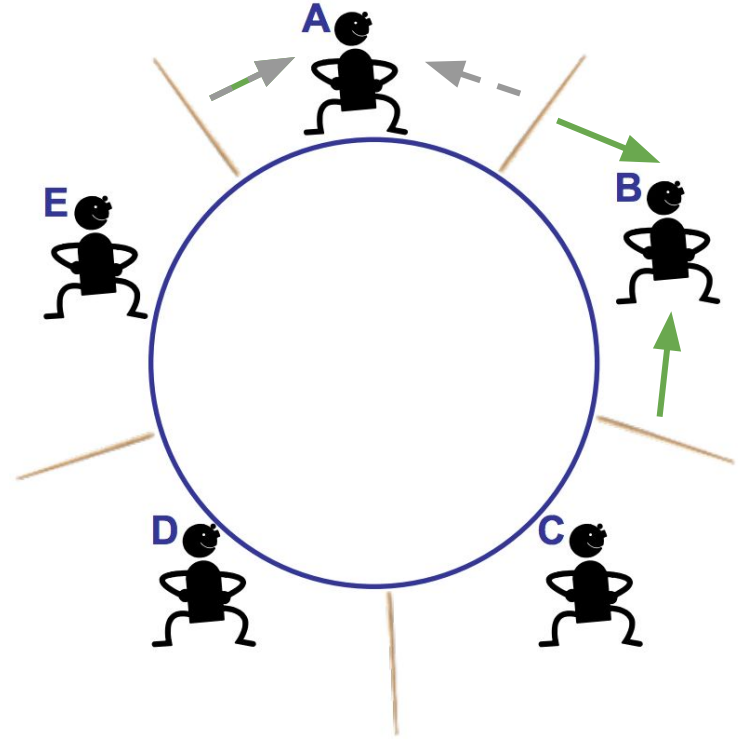
1. Threads grab all resources atomically, or none at all.



Preventing hold-and-wait

Two approaches:

2. If a thread finds a busy resource, it releases everything and tries again.



Deadlock exercise: bank account transfers

Suppose you have a function for transferring funds from one account to another.

Each user account has a lock.

You must acquire the lock for both accounts before transferring funds.

Deadlock exercise: bank account transfers

```
std::map<int, mutex> locks;
```

Can this deadlock?

```
void transfer_money(int acct1, int acct2, int amount) {  
    locks[acct1].lock();  
    locks[acct2].lock();  
    <transfer money>  
    locks[acct2].unlock();  
    locks[acct1].unlock();  
}
```

} Same issues as dining philosophers

Deadlock exercise: bank account transfers

Exercise 1:

Rewrite `transfer_money()` to be deadlock free, by attacking the **circular wait** condition.

Hint: How can you define an ordering on the locks?

Deadlock exercise: bank account transfers

```
std::map<int, mutex> locks;

void transfer_money(int acct1, int acct2, int amount) {
    locks[std::min(acct1, acct2)].lock();
    locks[std::max(acct1, acct2)].lock();
    <transfer money>
    locks[acct2].unlock();
    locks[acct1].unlock();
}
```

This part is fine!

Deadlock exercise: bank account transfers

Exercise 2:

Rewrite `transfer_money()` to be deadlock free, by attacking the **hold and wait** condition.

Hint: Find a way to acquire both locks **at once**.

Bonus: `std::scoped_lock`

- RAII lock management tool from the STL
 - automatically locks mutex(es) on construction, unlocks on destruction
- Can be constructed with **multiple** mutexes
- Built-in deadlock prevention mechanism

```
std::map<int, std::mutex> locks; // Or any Lockable type

void transfer_money(int acct1, int acct2, int amount) {
    std::scoped_lock lock{ locks[acct1], locks[acct2] };
    <transfer money>
}
```

Debugging Deadlock

Let's try it!

https://os.eecs.umich.edu/lab/semaphores_deadlocks.html

Two additional deadlock debugging tools

```
$ valgrind --tool=helgrind  
./deadlock
```

(does *not* require
recompilation)

<https://valgrind.org/docs/manual/hg-manual.html>

```
$ g++ -fsanitize=thread  
deadlock.cpp -o deadlock  
$ ./deadlock
```

(requires recompilation)

<https://clang.llvm.org/docs/ThreadSanitizer.html>

Project 2 Debugging

Project 2 Debugging

— — —

- Some bugs in P2 cannot be debugged simply using GDB
 - Your group may or may not struggle with a bug like these
 - If you do have them, they can be pretty tricky to address
 - We will introduce some ways to recognize and approach this class of bugs
- Disclaimer
 - What we present is **Non-Exhaustive!**
 - The slides represents a **tiny subset** of the possible issues and potential testing strategies for Project 2
 - You may be handling all things mentioned correctly and still have other errors that manifest in similar ways

What you might see when running the program

— — —

Segfault and/or “Corrupted Stack”

```
sarah sarahpc ~/Classes/eecs482/project2(main*) $ ./test2
Corrupted stack
Segmentation fault (core dumped)
```

*** stack smashing detected ***

```
ubuntu@ubuntu:~/Documents/coms352$ gcc add.c -o add
ubuntu@ubuntu:~/Documents/coms352$ ./add
Before add:
1
5
1Adding 4:
4
1
5
*** stack smashing detected ***: ./add terminated
Aborted (core dumped)
```

Screenshot from StackOverflow

What you might see in your debugger

— — —

“Cannot access memory” error

```
Thread 2 "test2" received signal SIGSEGV, Segmentation fault.  
[Switching to Thread 0x7ffff7a3a640 (LWP 12614)]  
munmap () at ../sysdeps/unix/syscall-template.S:122  
122      ../sysdeps/unix/syscall-template.S: No such file or directory.  
(gdb) bt  
#0  munmap () at ../sysdeps/unix/syscall-template.S:122  
Backtrace stopped: Cannot access memory at address 0x7ffff79b8d28
```

Question marks in backtrace (??)

```
Thread 2 "test2" received signal SIGSEGV, Segmentation fault.  
[Switching to Thread 0x7ffff7a3a640 (LWP 15365)]  
0x00005555555566516 in ?? ()  
(gdb) bt  
#0  0x00005555555566516 in ?? ()  
#1  0x00007ffff7b82130 in ?? () at ../sysdeps/unix/sysv/linux/x  
#2  0x0000000000000000 in ?? ()  
(gdb) █
```

Or other cryptic
output that you can't
easily make sense of

(Partial) Explanation

- The exact cause depends on the nature of your bug, but there are some frequent common causes
- Recall the 4 parts of a TCB, as discussed in Lab 4
 - stack ptr, registers, program counter, and stack
 - All four work together, so if there is a problem with any one, or an incongruence, **undefined behavior can occur**
- Think about these 4 parts and how they interact – what errors could occur from mishandling a *ucontext_t* or the associated variables?

Why is GDB/LLDB backtrace so unhelpful?

- Sadly, the debugger is not a magical problem diagnoser
- The `bt` command merely prints the frames of your current stack
 - For “simple” bugs, will take you to the problematic line of code
 - If you corrupt/delete your stack (for example), GDB cannot produce a coherent stack trace
- Instead of `bt`, use debugger to **step through** your code
 - Important to see execution line by line
 - Use `step` or `s` in GDB/LLDB

Print Debugging

- Print Debugging!!
 - Helpful for tracing the flow of execution in longer programs
 - Use copious print statements to trace down the line where the error occurs
 - Some good places to add print statements:
 - Whenever you handle a stack, especially at deletion
 - Anywhere you call `setcontext/swapcontext`
 - Start at the error and **“work backwards”**
 - Example: If you segfault when switching into a context, then work backwards to find the last place that context was modified.

Print Debugging

- Use `source_location` to show place of execution
 - 6cr: Perhaps include `cpu::self()` to show which CPU executed the line.
- On some IDEs (such as VS Code), formatting terminal output like `cpu.cpp:100` will produce a clickable link.
- Use in a macro with `printf`
 - Remember to avoid using `cout` in thread library code - can cause deadlock.

```
inline void printline(const std::source_location& location = std::source_location::current()) {  
    printf("CPU %p - Executed %s:%d\n", cpu::self(), location.file_name(), static_cast<int>(location.line()));  
}
```

```
• sarah sarahpc ~/Classes/eecs482/project2(main*) $ ./test2  
CPU 0x561902085e90 - Executed cpu.cpp:74  
CPU 0x561902085e90 - Executed cpu.cpp:79  
CPU 0x561902085e90 - Executed cpu.cpp:87
```

Print Debugging with #ifdef

Tired of losing AG submits because of stray print statements? Try this one simple trick!

In your makefile:

```
CC += -g -Wall -std=c++20 -O0 -DDEBUG
```

Now you can spam the macro to your heart's content :)

In your code:

```
#ifdef DEBUG
#define PRINTLINE printf("Helpful debugging stuff goes here\n");
#else
#define PRINTLINE
#endif
```


Questions?